

Introductions to Big Data Systems: Homework 2

Francisco Diego Ariel Romero Contreras

法兰西思科

2025403471

francisco.romero.c@ug.uchile.cl

Introduction

In this homework we were asked to study a parallelize solution using OpenMP of the known algorithm “page rank”.

```
void pageRank(Graph g, double *solution, double damping, double convergence)
```

The most remarkable parts to parallelize are two “for” loops. The first calculate the *score_new* and a value *broadcastScore*, which is a reduction. The second calculate the damped *score_new* and global difference, which is a reduction.

Solutions

calculate *score_new*

```
for (int i = 0; i < numNodes; ++i) {
    score_new[i] = 0.0;

    if (outgoing_size(g, i) == 0) {
        broadcastScore += score_old[i];
    }
    const Vertex *in_begin = incoming_begin(g, i);
    const Vertex *in_end = incoming_end(g, i);
    for (const Vertex *v = in_begin; v < in_end; ++v) {
        score_new[i] += score_old[*v] / outgoing_size(g, *v);
    }
    score_new[i] =
        damping * score_new[i] + (1.0 - damping) * equal_prob;
}
```

The principal thing to take attention is that *broadcastScore* is a reduction, it doesn't seems that way because is the first statement and not the unique, but this statement is not dependance of the other and also the other way. We could make the following notation to see it more clear

```
for (int i = 0; i < numNodes; ++i) {

    ...

    if (outgoing_size(g, i) == 0) {
        broadcastScore += score_old[i];
    }
}
```

Later we can make a reduction OpenMP sentence using the *broadcastScore* as reduction variable, as the operation is sum, the sentence is the following.

```

#pragma omp parallel for reduction(+:broadcastScore)
for (int i = 0; i < numNodes; ++i) {
    score_new[i] = 0.0;
    if (outgoing_size(g, i) == 0) {
        broadcastScore += score_old[i];
    }
    const Vertex *in_begin = incoming_begin(g, i);
    const Vertex *in_end = incoming_end(g, i);
    for (const Vertex *v = in_begin; v < in_end; ++v) {
        score_new[i] += score_old[*v] / outgoing_size(g, *v);
    }
    score_new[i] =
        damping * score_new[i] + (1.0 - damping) * equal_prob;
}

```

calculate *globalDiff*

```

for (int i = 0; i < numNodes; ++i) {
    score_new[i] += damping * broadcastScore * equal_prob;
    globalDiff += std::abs(score_new[i] - score_old[i]);
}

```

Here exist a dependency of *score_new* over *globalDiff*, and because of this dependency we can't calculate a reduction on both variables, so we can only reduce over *globalDiff*. And the OpenMP sentence is the following.

```

#pragma omp parallel for reduction(+:globalDiff)
for (int i = 0; i < numNodes; ++i) {
    score_new[i] += damping * broadcastScore * equal_prob;
    globalDiff += std::abs(score_new[i] - score_old[i]);
}

```

Performance

Table 1.

Graph	com-youtube_3m (1134890,2987624)	com-orkut_117m (3072441, 117185083)
Base (Reference)	0.165744	6.793877
one thread	0.165652	6.746097
two thread	0.133864	3.673797
Three thread	0.104092	3.073945
four thread	0.085160	2.475149

Table 1: Pagerank: Number of cores vs Graph nodes and Graph vertex Performance

Conclusions

The parallelization achieves a considerable reduction in execution time for graphs with a large number of edges and nodes, For instance, the runtime with four threads is reduced to nearly one third (0.3643205492239556) of the base execution time. The time reduction for four threads isn't a perfect quarter because the algorithmic can't be perfectly parallel. Certain segment of the code is sequential, later this will produce an efficiency lost because the Amdahl's Law.